

Launch Your Own AI Assistant

Zichao Yang

Zhongnan University of Economics & Law

Date: May 8, 2026

Roadmap

In this session, we will discuss three ways to launch your own AI assistant:

- Using browser-based LLM services
- Running smaller models locally
- Accessing frontier models through APIs
- AI Agents
- How to write a skill?

LLM via browser

All popular LLMs can be accessed through a browser on your computer. (*Note: not every company provides service in every region of the world.*)

China AI Products

- DeepSeek
- Kimi
- Qwen
- GLM
- MiniMax

U.S. AI Products

- ChatGPT
- Claude
- Gemini
- Grok
- Perplexity

Host Local LLMs

You can also host small size LLMs on your personal computers.

Choice 1: **Ollama** (Recommend)

Choice 2: **LM Studio**

Choice 3: **Ollama** + **AnythingLLM**

Local LLM + VSCode

Thanks to the extension **continue**, we can now use local LLMs inside VSCode, and make it “smater”.

To make this happen, we need to run **Ollama** in the background.

Large LLMs via APIs

To utilize cutting-edge LLMs, the most effective approach is to access them through their provided APIs.

In this guide, I will demonstrate how to use **Cherry Studio** to access the **DeepSeek models** hosted on ByteDance's **Volcano Engine** servers.

AI Agents

An AI agent is more than a chatbot: it is a system that can take a goal, plan a sequence of actions, use tools, and iteratively improve its output.

- **Input:** a task, a goal, or a user instruction
- **Reasoning:** break the task into manageable steps
- **Action:** use tools such as search, code execution, APIs, or local files
- **Iteration:** evaluate the result, revise, and continue until the task is completed

In practice, skills make AI agents more reliable by giving them task-specific instructions and reusable resources.

Popular AI Agents

- **Anthropic:** **Claude Code** (Claude Code Desktop, Claude CLI)
- **OpenAI:** **Codex** (Codex Desktop, Codex CLI)
- **Google:** **Gemini CLI**
- **Cursor:** **Cursor Agent**, access to all mainstream models.
- **GitHub:** **Copilot CLI** and **Copilot agent**
- **Open Source:** **OpenCode**

Best combo for me: VS Code + Extensions for the AI Agents above

What Is a Skill?

A skill is a modular instruction package that extends an AI agent with task-specific workflows, domain knowledge, and reusable resources.

What is a skill used for?

- Standardizing how the agent handles a recurring task
- Encoding domain-specific knowledge that is not obvious from the prompt alone
- Reusing scripts, references, and templates for reliable execution
- Reducing prompt-writing effort for complex or repetitive workflows

The Complete Guide to Building Skills for Claude

How to Write a Skill

- Create a folder with a required `SKILL.md` file
- Add YAML frontmatter with at least **name** and **description**
- In the body of `SKILL.md`, describe when the skill should be used and the workflow the agent should follow
- Put reusable code in `scripts/`, detailed documentation in `references/`, and templates or assets in `assets/` when needed

Shortcut: Forget all of the above and just ask your AI agent to write the skill for you!

In practice, you only need to provide two things: (1) the **task** and (2) the **expected outcome**. Then test the skill on a sample task, give **feedback**, and let the agent iteratively refine it until it meets your needs.